Programação para Dispositivos Móveis · Revisão

Simulado

Letícia Roberta Oliveira Souto

Marque uma alternativa por questão. Na correção, justifique por que as outras alternativas estão erradas.

1 Uma startup de entregas tem uma equipe pequena que domina **JavaScript** e precisa lançar o app para **Android e iOS em 3 meses**. O app usa câmera e GPS, mas não exige gráficos 3D nem processamento pesado. Qual abordagem é mais adequada?

A) Desenvolvimento nativo com Kotlin e Swift, garantindo máximo desempenho.

B) Desenvolvimento cross-platform com React Native, aproveitando o conhecimento da equipe.

C) Um site responsivo sem app, pois sites acessam câmera e GPS igual a apps nativos.

D) Desenvolvimento nativo apenas para Android, pois iOS não permite GPS.

2 Um app de academia calcula pontos de fidelidade. Analise o trecho em pseudocódigo. Qual valor será exibido?

```
pontos <- 0
para i de 1 ate 4 faca
   se i % 2 = 0 entao
      pontos <- pontos + i * 10
   senao
      pontos <- pontos + 5
   fimse
fimpara
escreva(pontos)
```

A) 40

B) 60

C) 70

D) 100

Um app de cantina recebe da API o JSON abaixo. Considerando que o JSON já foi convertido para a variável `dados`, qual expressão retorna o texto "Suco"?

```
{
  "pedido": 42,
  "itens": [
    { "nome": "Lasanha", "preco": 32.0 },
    { "nome": "Suco", "preco": 8.5 }
  ]
}
```

A) `dados.itens[2].nome`

B) `dados.itens[1].nome`

C) `dados.nome[1]`

D) `dados[1].itens.nome`

---

4 Um sistema de petshop possui as classes abaixo. O que será exibido e qual conceito explica o resultado?

```
class Animal {
    String emitirSom() { return "..."; }
}
class Cachorro extends Animal {
    @Override
    String emitirSom() { return "Au au"; }
}
// No programa principal:
Animal a = new Cachorro();
System.out.println(a.emitirSom());
```

A) "...", por encapsulamento.

B) "Au au", por polimorfismo.

C) "Au au", por sobrecarga de métodos.

D) Erro de compilação, pois a variável é do tipo Animal.

---

5 Em um app de cálculo de IMC, o usuário digita peso e altura, toca em "Calcular" e o resultado aparece abaixo. Qual alternativa classifica corretamente os elementos?

A) Entrada: Text do resultado; Processamento: TextInput; Saída: função calcularIMC.

B) Entrada: TextInput de peso e altura e o botão; Processamento: função calcularIMC; Saída: Text com o resultado.

C) Entrada: função calcularIMC; Processamento: botão; Saída: TextInput.

D) Todos os componentes visuais são saída, pois aparecem na tela.

6 Um aluno criou o código abaixo para um contador, mas ao tocar no botão o número na tela não muda. Qual é a causa do problema?

```
export default function Contador() {
  let total = 0;
  return (
    <View>
      <Text>{total}</Text>
      <Button title="+1" onPress={() => { total = total + 1; }} />
    </View>
  );
}
```

A) O componente Button não aceita a propriedade onPress.

B) total uma variável comum; a alteração não dispara nova renderização, o que exigiria `useState`.

C) Faltou usar `useEffect` com array vazio para atualizar o texto.

D) O valor deveria ser passado por `route.params`.

---

7 Um app de lista de compras precisa manter os itens do carrinho mesmo se o usuário fechar o app, sem internet e sem compartilhar com outros aparelhos. Os itens são uma lista simples de objetos. Qual solução é mais adequada?

A) Guardar os itens em `useState`, pois o estado é mantido pelo sistema operacional.

B) Enviar os itens para uma API remota a cada alteração.

C) Converter a lista para JSON e salvar com armazenamento local chave-valor, como AsyncStorage.

D) Criar um servidor MySQL dentro do aparelho.

---

8 No app da Cantina, o cliente finaliza um pedido e o app envia os itens para a API, que deve gravar um novo pedido no banco. A API responde indicando que o recurso foi criado. Qual combinação de método e status é esperada?

A) GET e 200

B) POST e 201

C) PUT e 404

D) DELETE e 500

---

9 Um app de academia quer avisar o aluno às 7h todos os dias para treinar. O lembrete não depende de nenhum dado do servidor. Qual solução é mais simples e adequada?

A) Notificação push enviada por um servidor dedicado todos os dias.

B) Notificação local agendada pelo próprio app, após o usuário conceder permissão.

C) Usar o GPS para detectar quando o aluno acorda.

D) Enviar um e-mail diário, pois apps não podem exibir notificações.

10 Analise o componente escrito em Kotlin com Jetpack Compose. Qual alternativa apresenta o equivalente em React Native?

```
@Composable
fun Saudacao(nome: String) {
    Text(text = "Olá, $nome")
}

// uso
Saudacao(nome = "Ana")
```

A) `function Saudacao({ nome }) { return <Text>Olá, {nome}</Text> }`, usado como `<Saudacao nome="Ana" />`

B) `function Saudacao() { return <Text>Olá, nome</Text> }`, usado como `<Saudacao />`

C) `<Text nome="Ana">Saudacao</Text>`

D) `class Saudacao { nome = "Ana" }`, sem retorno visual

11 Um app de biblioteca escolar exibe o catálogo vindo de uma API. Ao abrir a tela, os livros devem ser buscados uma única vez. Em React Native, onde a chamada deve ficar?

A) Diretamente no corpo do componente, antes do `return`.

B) Dentro do `onPress` de um botão "Atualizar".

C) Em um `useEffect` com array de dependências vazio.

D) Em um `useState`, passando a URL como valor inicial.

12 Em Java, a classe `ContaBancaria` tem o atributo `saldo` declarado como `private` e o método público `depositar(valor)` que só aceita valores positivos. Esse desenho aplica qual conceito?

A) Herança

B) Polimorfismo

C) Sobrecarga

D) Encapsulamento

13 Uma requisição `GET /produtos/999` retorna status 404. O que isso indica?

A) O recurso solicitado não foi encontrado no servidor.

B) O servidor sofreu uma falha interna.

C) O produto foi criado com sucesso.

D) O usuário não tem internet.

14 Qual comando SQL lista apenas os nomes dos produtos com estoque zerado?

A) `SELECT * FROM produto;`

B) `UPDATE produto SET estoque = 0;`

C) `DELETE FROM produto WHERE estoque = 0;`

D) `SELECT nome FROM produto WHERE estoque = 0;`

15 Em Kotlin com Jetpack Compose, qual anotação indica que uma função desenha uma parte da interface e pode ser reutilizada como componente?

A) `@Override`

B) `@Composable`

C)0 `@Test`

D) `@Entity`

---

16 Uma empresa precisa de um app com realidade aumentada pesada e uso intenso da câmera, apenas para iPhones corporativos, e tem uma equipe experiente em Swift. Qual abordagem é mais adequada?

A) PWA, por funcionar em qualquer navegador.

B) Cross-platform com React Native, pois é sempre a melhor escolha.

C) Nativo iOS com Swift.

D) Híbrido com WebView, por ser mais rápido de desenvolver.

Simulado · Parte 2

Questões dissertativas e de análise de código. Responda com suas palavras e, nas questões de código, mostre o raciocínio (teste de mesa), não apenas o resultado final.

17 ANÁLISE DE CÓDIGO · TESTE DE MESA

No app da Cantina, o carrinho calcula o total do pedido e aplica 10% de desconto em compras a partir de R$ 40. Analise o código.

```
const carrinho = [
  { nome: "Coxinha",       preco: 6, qtd: 3 },
  { nome: "Refrigerante", preco: 7, qtd: 2 },
  { nome: "Brownie",       preco: 9, qtd: 2 },
];

let total = 0;
for (let i = 0; i < carrinho.length; i++) {
  total = total + carrinho[i].preco * carrinho[i].qtd;
}

if (total >= 40) {
  total = total * 0.9;
}
console.log(total);
```

a) Faça o teste de mesa mostrando o valor de `i` e de `total` ao final de cada volta do laço.

b) Qual valor é exibido no console?

c) Se o cliente remover os Brownies do carrinho, o desconto ainda será aplicado? Justifique com o cálculo.

b) O total é de R$50,00. Como ultrapassa R$40,00 é aplicado 10% de desconto.

R$50,00 - R$5,00 = R$45,00

d) Não, sem os brownies o total será R$ 18,00 + R$ 14,00 = R$ 32,00. Como R$ 32,00 é menor que R$40,00, o desconto não será aplicado.

a)

Volta	Produto	Cálculo	Total
1	Coxinha	0 + 6 × 3	R$ 18,00
2	Refrigerante	18 + 7 × 2	R$ 32,00
3	Brownie	32 + 9 × 2	R$ 50,00

18 **ANÁLISE DE CÓDIGO · ENCONTRE O ERRO**

Um app de quiz compara as respostas do aluno com o gabarito para contar os acertos. O aluno acertou 2 questões, mas o app mostrou outro número.

```
const respostas = ["B", "B", "C", "D"];
const gabarito  = ["A", "B", "B", "D"];

let acertos = 0;
for (let i = 1; i <= respostas.length; i++) {
  if (respostas[i] === gabarito[i]) {
    acertos++;
  }
}
console.log("Acertos: " + acertos);
```

a) Qual valor o código exibe? Mostre o que acontece em cada volta.

b) Explique os **dois** problemas da linha do `for` e por que eles levam ao resultado errado.

c) Reescreva a linha do `for` corrigida.

---

b) Começa em i = 1: pula a primeira posição do array, que é o índice 0.

Usa i <= respostas.length: permite acessar uma posição além do final do array, causando uma comparação com undefined.

c) for (let i = 0; i < respostas.length; i++) {

---

a)

Volta	Comparação	Acertos
1	B ≠ B? Não, são iguais	1
2	B = B	2
3	C ≠ B	2
4	D = D	3

A tela de cadastro de um app chama a função abaixo ao tocar em "Criar conta" e exibe a mensagem retornada.

```
function validarCadastro(nome, email, senha) {
  if (nome.length < 3) {
    return "Nome muito curto";
  }
  if (!email.includes("@")) {
    return "E-mail inválido";
  }
  if (senha.length < 6) {
    return "Senha fraca";
  }
  return "OK";
}
```

a) Complete a tabela com a mensagem exibida em cada caso:

Chamada	Mensagem exibida
`validarCadastro("Ana", "ana@mail.com", "123")`	Senha fraca
`validarCadastro("Jo", "jo.com", "12")`	Nome muito curto
`validarCadastro("Bruno", "bruno.com", "123456")`	E-mail inválido
`validarCadastro("Carla", "carla@mail.com", "abc123")`	OK

b) Na segunda chamada existem três dados inválidos, mas o usuário vê só uma mensagem. Explique por quê, considerando o funcionamento do `return`, e diga se isso é bom ou ruim para a experiência do usuário.

O return encerra a função assim que encontra o primeiro erro, por isso as outras validações não são executadas. Isso pode ser ruim para a experiência do usuário, pois ele precisa corrigir um erro de cada vez. O ideal seria mostrar todos os erros de uma vez.

No app do Petshop, a tela de agenda usa os dados abaixo. Os resultados de `filter` e `map` alimentam uma `FlatList` .

```
const agendamentos = [
  { pet: "Thor", servico: "banho",     valor: 50 },
  { pet: "Mel",  servico: "tosa",      valor: 70 },
  { pet: "Bob",  servico: "banho",     valor: 50 },
  { pet: "Luna", servico: "consulta", valor: 120 },
];

const banhos = agendamentos.filter(a => a.servico === "banho");
const nomes  = banhos.map(a => a.pet);
```

a) Qual o valor de `banhos.length` ? E o conteúdo de `nomes` ?

b) A lista `agendamentos` original foi alterada pelo `filter` ? Explique.

c) Escreva um trecho de código, usando um laço, que calcule o faturamento total do dia (soma de todos os valores).

a) banhos.lenght: 2 | nomes:[“Thor”, “Bob”]

b) Não. O filter() cria uma lista com os elementos que atendem à condição, sem alterar a lista original.

c) let total = 0;

for (let i = 0; i < agendamentos.length; i++) {
  total += agendamentos[i].valor;
}

console.log(total); // 310

Um app bancário bloqueia a conta após 3 senhas erradas. O código simula as senhas digitadas por um usuário, em ordem.

```
const senhasDigitadas = ["1234", "abcd", "senha", "9876"];
const senhaCorreta = "9876";

let tentativas = 0;
let bloqueado = false;
let i = 0;

while (i < senhasDigitadas.length && !bloqueado) {
  if (senhasDigitadas[i] === senhaCorreta) {
    console.log("Login liberado");
    break;
  }
  tentativas++;
  if (tentativas === 3) {
    bloqueado = true;
    console.log("Conta bloqueada");
  }
  i++;
}
```

a) O que é exibido no console? Mostre os valores de `i`, `tentativas` e `bloqueado` a cada volta.

b) A 4ª senha digitada é a correta. Por que ela nunca é verificada?

c) Do ponto de vista de segurança, esse comportamento está correto? Justifique.

b) Porque, após 3 tentativas erradas, bloqueado passa a ser true, encerrando o while. Assim, a 4ª senha não é verificada.

c) Sim. Bloquear a conta após 3 senhas erradas ajuda a impedir tentativas repetidas de descobrir a senha.

---

a)

Volta	i	tentativas	bloqueado	Console
1ª	0	1	false	—
2ª	1	2	false	—
3ª	2	3	true	Conta bloqueada

22 ANÁLISE DE CÓDIGO · ESTADO E RENDERIZAÇÃO

Uma tela de produto tem um botão de favoritar e um contador de curtidas.

```
export default function Produto() {
  const [favorito, setFavorito] = useState(false);
  const [curtidas, setCurtidas] = useState(10);

  function alternar() {
    setFavorito(!favorito);
    setCurtidas(favorito ? curtidas - 1 : curtidas + 1);
  }

  return (
    <View>
      <Text>{curtidas} curtidas</Text>
      <Button
        title={favorito ? "Remover dos favoritos" : "Favoritar"}
        onPress={alternar}
      />
    </View>
  );
}
```

a) Complete a tabela com o que aparece na tela:

Momento	Texto do botão	Curtidas exibidas
Ao abrir a tela	Favoritar	10
Após o 1º toque	Remover dos favoritos	11
Após o 2º toque	Favoritar	10
Após o 3º toque	Remover dos favoritos	11

b) Explique como o operador ternário (`? :`) decide o texto do botão e por que a tela é atualizada a cada toque.

O operador ternário (? :) verifica se favorito é verdadeiro. Se for, mostra "Remover dos favoritos"; caso contrário, mostra "Favoritar". A tela é atualizada a cada toque porque setFavorito e setCurtidas alteram os estados, fazendo o React Native renderizar a tela novamente.

23 ANÁLISE DE CÓDIGO · ORDEM DE EXECUÇÃO

Ao abrir a tela de cardápio da Cantina, o app executa o código abaixo.

```
console.log("1 - abrindo tela");

fetch("https://api.cantina.com/cardapio")
  .then(resposta => resposta.json())
  .then(dados => console.log("2 - itens recebidos: " + dados.length));

console.log("3 - tela desenhada");
```

a) Em que ordem as mensagens aparecem no console? Explique o motivo.

b) O que o usuário vê na tela entre a mensagem "3" e a mensagem "2"? Que recurso de interface você colocaria nesse intervalo e com qual variável de estado o controlaria?

a) 1. "1 - abrindo tela" | 2. "3 - tela desenhada" | 3. "2 - itens recebidos"

Motivo: o fetch é assíncrono, então a tela é desenhada antes da resposta da API chegar.

b) O usuário vê a tela sem os itens do cardápio carregados.

Eu colocaria um indicador de carregamento (loading), controlado por uma variável de estado, como carregando (true enquanto os dados são buscados e false quando chegam).

---

24 DISSERTATIVA · ESCREVA A LÓGICA

O app da Cantina mostra ao cliente o status do pedido com base em quantos minutos se passaram desde que ele foi feito.

Minutos desde o pedido	Status exibido
Menor que 0	"Tempo inválido"
De 0 a 4	"Pedido recebido"
De 5 a 14	"Em preparo"
15 ou mais	"Pronto para retirada"

a) Escreva a função `statusPedido(minutos)`, em JavaScript ou pseudocódigo, que retorne o texto correto.

b) Indique o retorno para os valores 4, 5, 15 e -2. Por que é importante testar justamente esses valores?

b) Valor | Retorno

4	Pedido recebido
5	Em preparo
15	Pronto para retirada
-2	Tempo inválido

a)
```
function statusPedido(minutos) {
if (minutos < 0) {
return "Tempo inválido";
} else if (minutos < 5) {
  return "Pedido recebido";
  } else if (minutos < 15) {
    return "Em preparo";
  } else {
    return "Pronto para retirada";
    }
 }
```

25 **DISSERTATIVA · RACIOCÍNIO PASSO A PASSO**

O carrinho da Cantina é salvo no AsyncStorage como uma lista de objetos `{ id, nome, qtd }`. Quando o cliente toca em "Adicionar" num item que já está no carrinho, a quantidade deve aumentar em vez de aparecer o item duplicado.

a) Descreva, passo a passo (em português ou pseudocódigo), toda a lógica do botão "Adicionar": desde ler o carrinho salvo até gravá-lo de volta. Considere também o caso de o carrinho ainda não existir.

b) Por que é necessário usar `JSON.parse` e `JSON.stringify` nesse processo?

a) 1.Ler o carrinho salvo no AsyncStorage.
2.Se o carrinho não existir, criar uma lista vazia.
3.Procurar se o item já existe no carrinho pelo id.
4.Se existir, aumentar sua quantidade (qtd) em 1.
5.Se não existir, adicionar o item à lista com quantidade 1.
6.Converter a lista em texto e salvar novamente no AsyncStorage.

b) - JSON.parse transforma o texto salvo no AsyncStorage em uma lista de objetos que o JavaScript consegue manipular.

- JSON.stringify transforma a lista de objetos em texto para que possa ser salva no AsyncStorage.